Adapter Pattern for Music Band

:Prepared by

- Khader Al-Nuble (Leader)
- Yousef Hariri
- Yahya Khafeer
- Samer Al-Jaafari

Agenda

- Real-World Problem & Challenges
- Pattern Comparison & Trade-offs
- Selected Pattern & Justification (SOLID)
- UML Diagram & Technical Documentation
- Implementation (Code Structure)
- Demo
- Team Contributions
- GitHub Project Management
- Q&A

Real-World Software Problem

- **Scenario:** Music band management system needs to control different instruments.
- **Problem:** Each instrument has its own unique method to produce sound:
- ElectricGuitar → startElectric() + playSound()
- ElectricPiano → startElectric() + playSound()
- Flute → blow()
- Drum → hit()
- **Goal:** A musician/conductor wants to play any instrument using a single unified command: **play()**

Main Design Challenges

Interface incompatibility between existing classes.

Open for Extension: Must support adding new instruments (saxophone, violin) without modifying client code.

Decoupling: Client (musician) should depend on abstraction (Playable), not concrete classes.

Pattern Comparison (Table)

Criteria	Adapter Pattern	Facade Pattern
Intent	Convert interface	Simplified interface
Solves	Interface mismatch	System complexity
New Instruments	✅ Easy (New Adapter)	❌ Hard (Modify Facade)
Best for	Unifying interfaces	Simplifying subsystems

Trade-offs & Final Decision

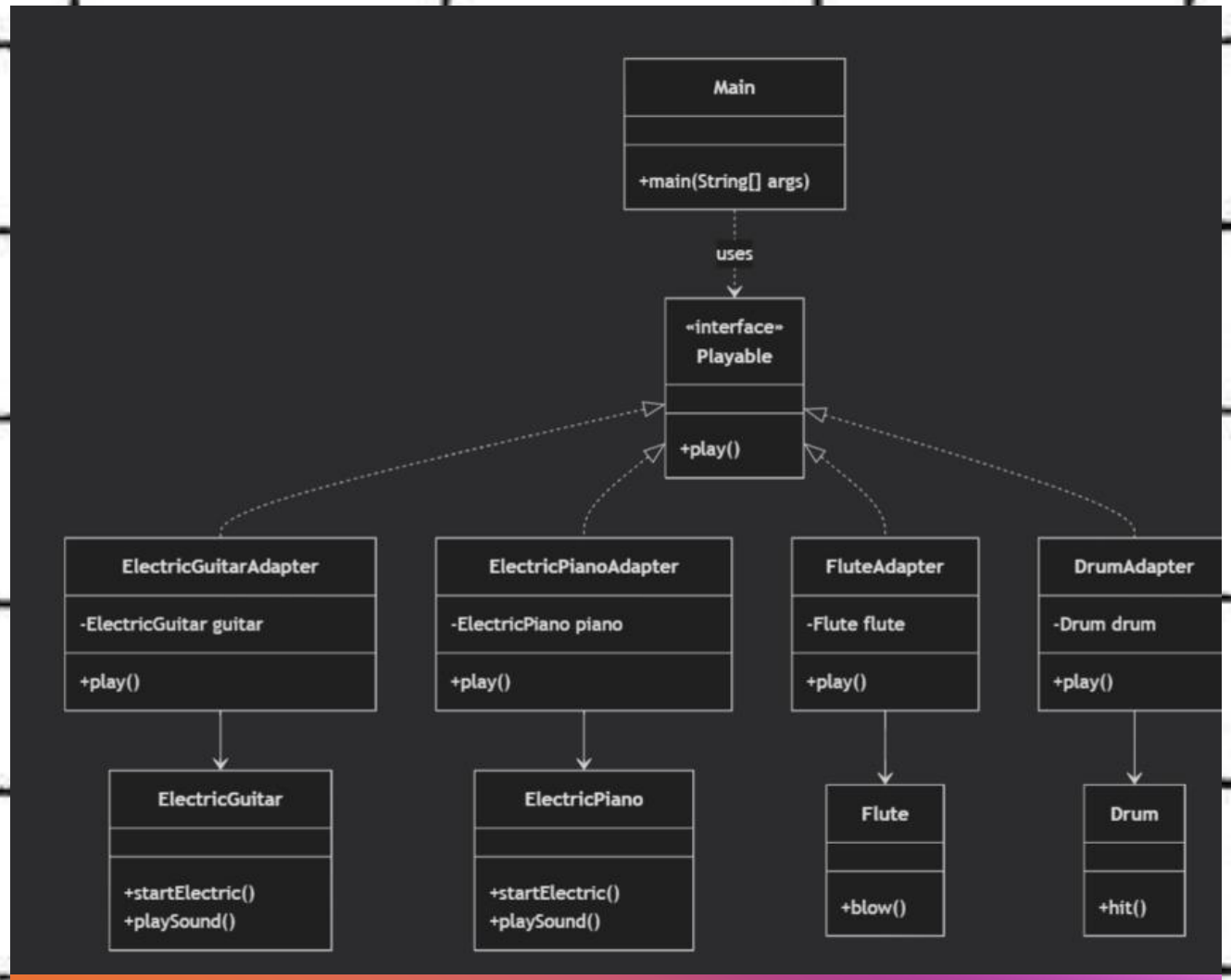
- **Final Decision:** ✅ **Adapter Pattern**
- **Advantages:** Supports Open/Closed Principle, Single Responsibility, and works with legacy code.
- **Disadvantages:** Increases number of classes, slight delegation overhead.

Selected Pattern & Justification (SOLID)

- **SRP:** Each adapter handles only one instrument type.
- **OCP:** System open for new adapters, closed for modification.
- **ISP:** Target interface *Playable* has only *play()*.
- **DIP:** Client depends on *Playable* abstraction.
- **Variant:** Object Adapter (uses composition)

UML Diagram

- **Elements:**
- **Interface:** Playable (+play)
- **Adapters:** GuitarAdapter, PianoAdapter, etc.
- **Adaptees:** ElectricGuitar, Drum, etc.



Applying the Pattern Step-by-Step

- Define **Playable** interface.
- Existing instrument classes (Adaptees) remain unchanged.
- Create an **Adapter** for each instrument.
- Adapter translates `play()` to the specific method (e.g., `hit()`).
- Client uses only `Playable` references.

```
src/
├── Playable.java                    (Target interface)
├── adaptees/
│   ├── ElectricGuitar.java
│   ├── ElectricPiano.java
│   ├── Flute.java
│   └── Drum.java
├── adapters/
│   ├── ElectricGuitarAdapter.java
│   ├── ElectricPianoAdapter.java
│   ├── FluteAdapter.java
│   └── DrumAdapter.java
└── Main.java                        (Client)
```

Code Structure

GitHub Project Management

Project Link: SDejrboqt/music-adapter-pattern

Task Tracking: Issues and milestones used for analysis, UML, and code.

Evidence: Commit history and Project Board active.

Team Contributions

Member Name	Role	Specific Contribution
Khader Al-Nuble (Leader)	Project Leader –	• Core idea & problem definition • All source code • Complete presentation slides • Presentation script (scenario) • Final presentation recording • GitHub setup & management • Final project packaging
Yousef Hariri	Problem Analysis	• Wrote problem statement & design challenges • Compared Adapter vs. Facade patterns • Documented trade-offs
Yahya Khafeer	Pattern Justification	• Selected Adapter pattern • Wrote SOLID justification • Contributed to Justification Document
Samer Al-Jaafari	UML & Documentation	• Created UML class diagram • Described relationships & pattern application • Wrote technical documentation (README structure)

Summary of Deliverables

- ✅ Problem Analysis & Trade-offs Report
- ✅ Pattern Selection & Justification Document
- ✅ UML Diagram + Technical Documentation
- ✅ Source Code + README
- ✅ GitHub Project Board
- ✅ Contribution Log (Commit history)
- ✅ Final Presentation Slides + Recording
- ✅ Integrated Project Package